

Large Language Models
for the
Economic and Social Sciences:
Demystifying LLMs II

Indira Sen, HWS 2025

<https://github.com/dess-mannheim/LLM4ESS-HWS25>

contents

Embeddings

language modeling, perplexity

Deep learning

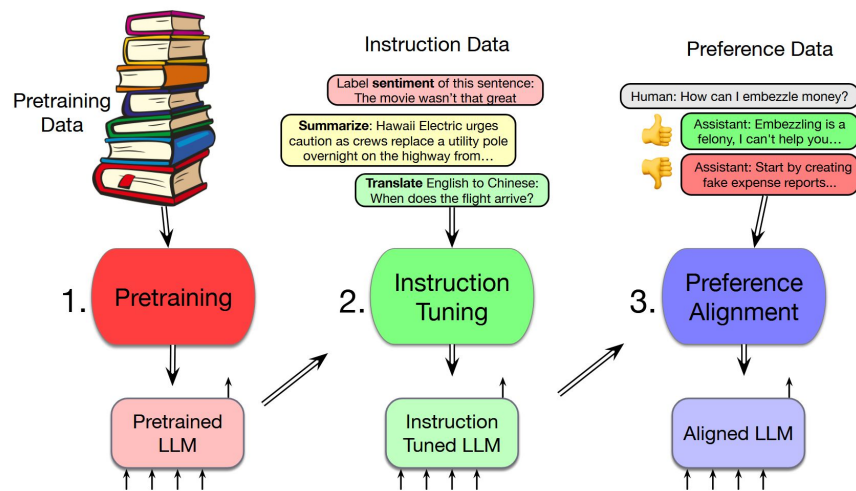
Attention

Contextual embeddings

transformers

Last Time: How can we build LLMs?

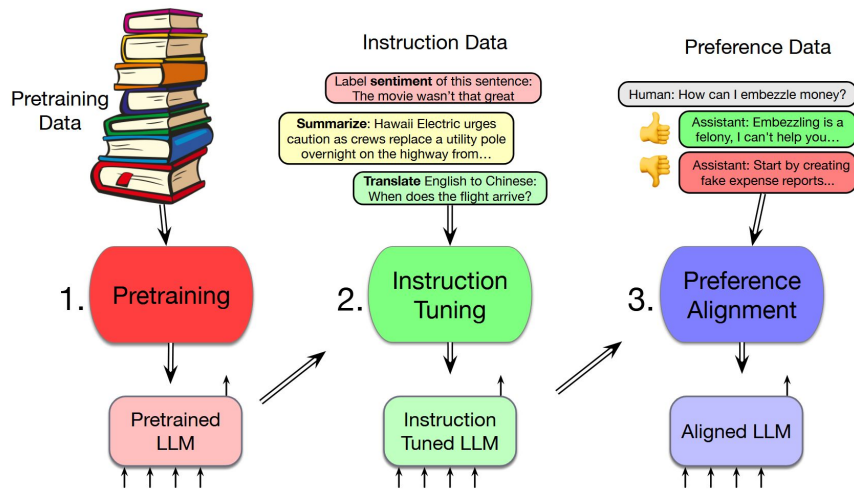
Somewhat simplified answer



Jurafsky and Martin, 2025

Last Time: How can we build LLMs?

Somewhat simplified answer



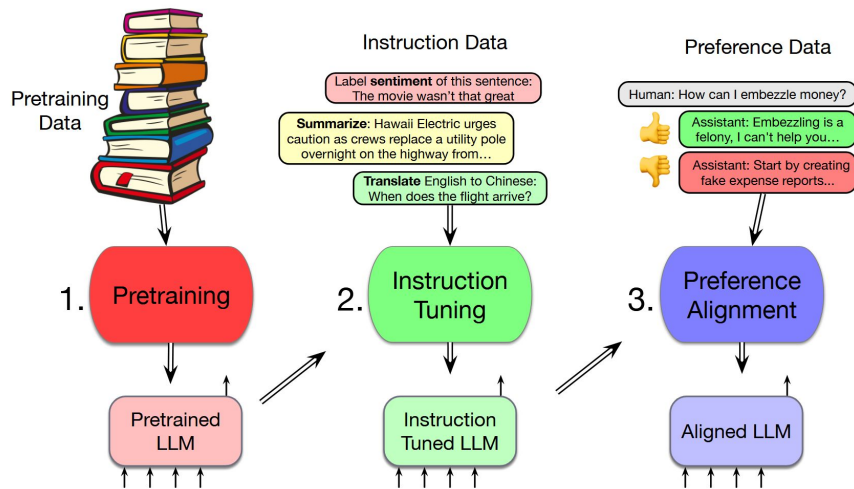
Jurafsky and Martin, 2025

Vastly Simplified answer

- We take a LOT of data encoding social, cultural, and of course linguistic traces
- Train NLP models that can generate content on this data
- We further train the model to follow and respond to instructions
- We further modify the model so that it doesn't have unwanted behavior

Last Time: How can we build LLMs?

Somewhat simplified answer



Jurafsky and Martin, 2025

Vastly Simplified answer

- We take a LOT of data encoding social, cultural, and of course linguistic traces
- **Train NLP models that can generate content on this data**
- We further train the model to follow and respond to instructions
- We further modify the model so that it doesn't have unwanted behavior

Summary of Last Week and Today

Last session we learned:

- About the computational turn in the Social Sciences
- About LLMs and how they contribute to the Social Sciences
- The first building blocks of LLMs
 - Words → Tokens
 - Text Representations
 - Using text representations in text classification

Today:

- More efficient representations → word embeddings
- More powerful models for classification → deep learning
- Doing more with text representations → language modeling
- Attention and Transformers
- More about course projects and how they will be graded

Better Representations of Text

We saw last time the
bag-of-words
representation

Document D1	<i>The child makes the dog happy</i> the: 2, dog: 1, makes: 1, child: 1, happy: 1
Document D2	<i>The dog makes the child happy</i> the: 2, child: 1, makes: 1, dog: 1, happy: 1



	child	dog	happy	makes	the	BoW Vector representations
D1	1	1	1	1	2	[1,1,1,1,2]
D2	1	1	1	1	2	[1,1,1,1,2]

a list of numbers: Vector

a list of lists
Matrix

What's wrong with Bag-of-words representation?

- Language is highly contextual with a specific flow
- Bag-of-words ignores order and how words are related to each other
- But, these signals are important!
- Also, extremely sparse representation. If we have 10K documents with an average vocab of 8K tokens, we end up with 10K vectors of 8K length, most of which is 0

“Mannheim is a beautiful ...”

city

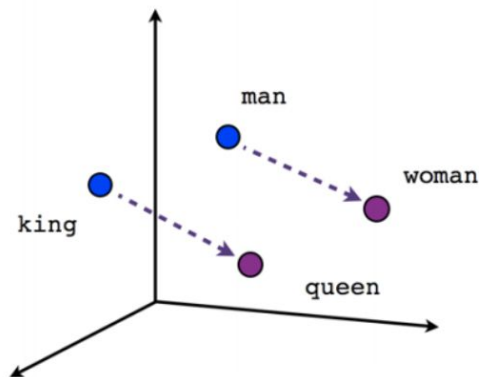
in

place

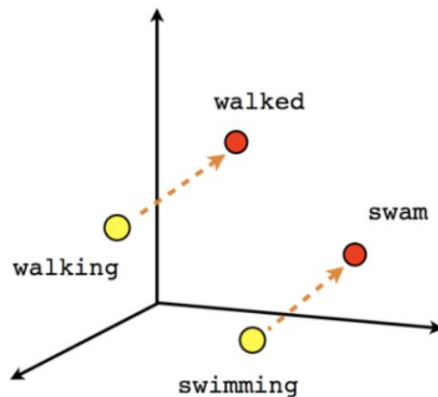
water

example

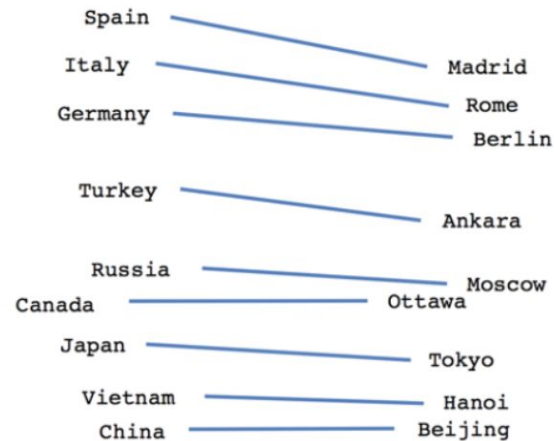
Vector Semantics



Male-Female



Verb tense



Country-Capital

Relations captured by word2vec. (Source: [NLP's ImageNet Moment](#))

Vector Semantics

“ As Wittgenstein says, ‘the meaning of words lies in their use.’ The day-to-day practice of playing language games recognizes customs and rules. It follows that a text in such established usage may contain sentences such as ‘Don’t be such an ass!’, ‘You silly ass!’, ‘What an ass he is!’ In these examples, the word ass is in familiar and habitual company, commonly collocated with you silly—, he is a silly—, don’t be such an—. **You shall know a word by the company it keeps!** ”

- John Rupert Firth, “A Synopsis of Linguistic Theory” 1957

Words have senses

- One word “bank” can have many meanings:

Sense 1: ■ ...a **bank** can hold the investments in a custodial account...

Sense 2: ■ “...as agriculture burgeons on the east **bank** the river will shrink even more”

- **Sense** (or **word sense**)
 - A discrete representation
of an aspect of a word’s meaning.
- The word **bank** here has two senses

Handcrafted Resources: WordNet 3.0

- A hierarchically organized lexical database
- Online thesaurus + aspects of a dictionary
 - Some [other languages](#) available or under development
 - (Arabic, Finnish, German, Portuguese...)

Category	Unique Strings
Noun	117,798
Verb	11,529
Adjective	22,479
Adverb	4,481

Word Similarity

- **Synonymy**: a binary relation
 - Two words are either synonymous or not
- **Similarity** (or **distance**): a looser metric
 - Two words are more similar if they share more features of meaning
- Similarity is properly a relation between **senses**
 - The word “bank” is not similar to the word “slope”
 - Bank¹ is similar to fund³
 - Bank² is similar to slope⁵

Two classes of similarity algorithms

- Thesaurus-based algorithms
 - Are words “nearby” in hypernym hierarchy?
 - Do words have similar glosses (definitions)?
- Distributional algorithms
 - Do words have similar distributional contexts?

Problems with thesaurus-based meaning

- We don't have a thesaurus for every language
- Even if we do, they have problems with **recall**
 - Many words are missing
 - Most (if not all) phrases are missing
 - Some connections between senses are missing
 - Thesauri work less well for verbs, adjectives
 - Adjectives and verbs have less structured hyponymy relations

Distributional models of meaning

- Also called vector-space models of meaning
- Offer much higher recall than hand-built thesauri
 - Although they tend to have lower precision
- Zellig Harris (1954): “**oculist** and **eye-doctor** ... occur in almost the same environments....
If A and B have almost identical environments we say that they are synonyms.
- Firth (1957): “You shall know a word by the company it keeps!”

Intuition of distributional word similarity

- Example:

A bottle of **tesgüino** is on the table
Everybody likes **tesgüino**
Tesgüino makes you drunk
We make **tesgüino** out of corn.

- From context words humans can guess **tesgüino** means
 - an alcoholic beverage like **beer**
- Intuition for algorithm:
 - Two words are similar if they have similar word contexts.

Term-document matrix

- Each cell: count of term t in a document d : $tf_{t,d}$
 - Each document is a **count vector** in $\mathbb{N}^V$: a column below

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	1	8	15
soldier	2	2	12	36
fool	37	58	1	5
clown	6	117	0	0

Term-document matrix

- Two documents are similar if their vectors are similar

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	1	8	15
soldier	2	2	12	36
fool	37	58	1	5
clown	6	117	0	0

The words in a term-document matrix

- Each word is a **count vector** in $\mathbb{N}^D$: a row below

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	1	8	15
soldier	2	2	12	36
fool	37	58	1	5
clown	6	117	0	0

The words in a term-document matrix

- Two **words** are similar if their vectors are similar

	As You Like It	Twelfth Night	Julius Caesar	Henry V
battle	1	1	8	15
soldier	2	2	12	36
fool	37	58	1	5
clown	6	117	0	0

The Term-Context matrix

- Instead of using entire documents, use smaller contexts
 - Paragraph
 - Window of 10 words
- A word is now defined by a vector over counts of context words

Term-context matrix for word similarity

- Two **words** are similar in meaning if their context vectors are similar

	aardvark	computer	data	pinch	result	sugar	...
apricot	0	0	0	1	0	1	
pineapple	0	0	0	1	0	1	
digital	0	2	1	0	1	0	
information	0	1	6	0	4	0	

Should we use raw counts?

- For the term-document matrix
 - We used **tf-idf** instead of raw term counts
- For the term-context matrix
 - **Positive Pointwise Mutual Information (PPMI)** is common

TF-IDF: Term Frequency-Inverse Document Frequency

A transformation to represent a term as function of how frequently it appears in a document but accounting for its frequency in the overall dataset

- Term Frequency (TF) for a given term = the number of times the term appears
- Document Frequency (DF) for a given term = the number of documents in collection / number of documents that contain term
- $TF\text{-}IDF = TF/DF = TF * \textbf{inverse DF}$

Formalizing

- Term frequency ($tf_{t,d}$) = the number of times term t occurs in document d ;
- Inverse document frequency = inverse fraction of number of documents containing (D_t) among total number of documents N

$$tfidf(t, d) = tf_{t,d} \times \log \frac{N}{D_t}$$

Pointwise Mutual Information

- **Pointwise mutual information:**

- Do events x and y co-occur more than if they were independent?

$$\text{PMI}(X, Y) = \log_2 \frac{P(x, y)}{P(x)P(y)}$$

- **PMI between two words:**

(Church & Hanks 1989)

- Do words x and y co-occur more than if they were independent?

$$\text{PMI}(\text{word}_1, \text{word}_2) = \log_2 \frac{P(\text{word}_1, \text{word}_2)}{P(\text{word}_1)P(\text{word}_2)}$$

Is any of this still relevant?

- Find distinctive words using TF-IDF and PPMI

Analyzing Polarization in Social Media: Method and Application to Tweets on 21 Mass Shootings

Dorottya Demszy¹ Nikhil Garg¹ Rob Voigt¹ James Zou¹

Matthew Gentzkow¹ Jesse Shapiro² Dan Jurafsky¹

¹Stanford University ²Brown University

{ddemszky, nkgarg, robvoigt, jamesz, gentzkow, jurafsky}@stanford.edu
jesse.shapiro.1@brown.edu

Abstract

We provide an NLP framework to uncover four linguistic dimensions of political polarization in social media: topic choice, framing, affect and illocutionary force. We quantify these aspects with existing lexical methods, and propose clustering of tweet embeddings as a means to identify salient topics for analysis across events; human evaluations show that our approach generates more cohesive topics than traditional LDA-based models. We apply our methods to study 4.4M tweets on 21 mass shootings. We provide evidence that the discussion of these events is highly polarized

2016) and Facebook (Bakshy et al., 2015). Prior NLP work has shown, e.g., that polarized messages are more likely to be shared (Zafar et al., 2016) and that certain topics are more polarizing (Balasubramanyan et al., 2012); however, we lack a more broad understanding of the many ways that polarization can be instantiated linguistically.

This work builds a more comprehensive framework for studying linguistic aspects of polarization in social media, by looking at topic choice, framing, affect, and illocutionary force.

1.1 Mass Shootings

Latent Semantic Analysis (LSA)

- a technique analyzing relationships between a **set of documents** and **the terms they contain** by producing a set of concepts related to the documents and terms.
- Simplified description:
 - Find the term-document matrix
 - Apply Singular Value Decomposition

[Why?]

Latent Semantic Analysis (LSA)

- a technique analyzing relationships between a **set of documents** and **the terms they contain** by producing a set of concepts related to the documents and terms.
- Simplified description:
 - Find the term-document matrix
 - Apply Singular Value Decomposition
- For dimensionality reduction
- https://en.wikipedia.org/wiki/File:Topic_model_scheme.webm
- We can now use cosine similarity (or other similarity metrics) to find the similarity between documents!

[Why?]

Practical Uses

LSA: Topic modeling, similarity between documents

Finding distinctive or discriminative words in subsets of data

Example: How is hate speech and toxicity defined in different languages in current datasets?



TF-IDF, LSA are count-based representations. Good, but still sparse

Can we learn dense representations instead?

Learn Representations of Words

- So far: all methods based on counting; no learning or prediction

- Now: Learn low-dimensional **dense** representations ('vectors') through prediction: **using context to predict words in a surrounding window**

- Transform this into a supervised prediction problem

Word2Vec

- **Learned parameters:** word vectors
- **Goal:** make each vector “know” about the contexts of its word
- **How:** train vectors to predict possible contexts from words (or, alternatively, words from contexts)

Important: These are word *type* vectors

Distributed Representations of Words and Phrases and their Compositionality

Tomas Mikolov
Google Inc.
Mountain View
mikolov@google.com

Ilya Sutskever
Google Inc.
Mountain View
ilyasu@google.com

Kai Chen
Google Inc.
Mountain View
kai@google.com

Greg Corrado
Google Inc.

Jeffrey Dean
Google Inc.

Efficient Estimation of Word Representations in Vector Space

Tomas Mikolov
Google Inc., Mountain View, CA
tmikolov@google.com

Kai Chen
Google Inc., Mountain View, CA
kaichen@google.com

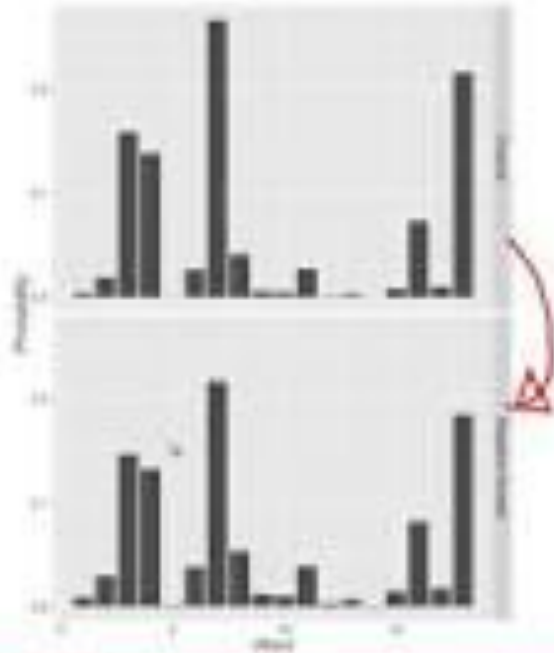
Greg Corrado
Google Inc., Mountain View, CA
gcorrado@google.com

Jeffrey Dean
Google Inc., Mountain View, CA
jeff@google.com

Abstract

We propose two novel model architectures for computing continuous vector representations of words from very large data sets. The quality of these representations is measured in a word similarity task, and the results are compared to the previously best performing techniques based on different types of neural networks. We observe large improvements in accuracy at much lower computational cost, i.e. it takes less than a day to learn high quality word vectors from a 1.6 billion words data set. Furthermore, we show that these vectors provide state-of-the-art performance on our test set for measuring syntactic and semantic word similarities.

Negative Sampling Distribution



$$p^{NS}(w) = \frac{f(w)}{\sum_w f(w)} \quad (3)$$

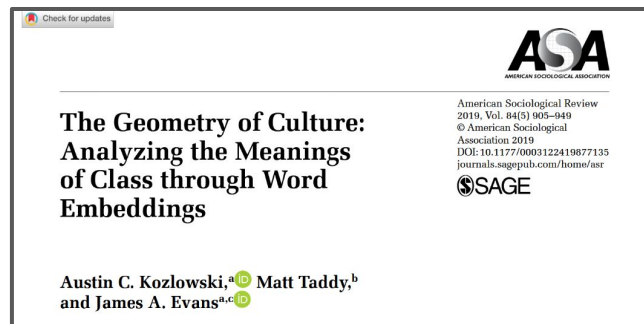
Brings down frequent terms, brings up infrequent terms



<https://www.youtube.com/watch?v=QyrUentbkvw>

Applications of word embeddings in the Social Sciences

- [Kozlowski et al.](#): Assess culture and class and how language encodes it through time
- [Garg et al.](#): Compute a gender or ethnic bias for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities



Abstract

We argue word analysis of semantic relationships between words in text from multiple dimensions of the basic cultural, educational, and cultivated

Word embeddings quantify 100 years of gender and ethnic stereotypes

Nikhil Garg^{a,1}, Londa Schiebinger^b, Dan Jurafsky^{c,d}, and James Zou^{a,f,1}

^aDepartment of Electrical Engineering, Stanford University, Stanford, CA 94305; ^bDepartment of History, Stanford University, Stanford, CA 94305; ^cDepartment of Linguistics, Stanford University, Stanford, CA 94305; ^dDepartment of Computer Science, Stanford University, Stanford, CA 94305; ^eDepartment of Biomedical Data Science, Stanford University, Stanford, CA 94305; and ^fChan Zuckerberg Biohub, San Francisco, CA 94158

Edited by Susan T. Fiske, Princeton University, Princeton, NJ, and approved March 12, 2018 (received for review November 22, 2017)

Word embeddings are a powerful machine-learning framework that represents each English word by a vector. The geometric relationship between these vectors captures meaningful semantic relationships between the corresponding words. In this paper, we develop a framework to demonstrate how the temporal dynamics of the embedding helps to quantify changes in stereotypes and attitudes toward women and ethnic minorities in the 20th and 21st centuries in the United States. We integrate word embeddings trained on 100 y of text data with the US Census to show that changes in the embedding track closely with demographic and occupation shifts over time. The embedding captures societal shifts—e.g., the women's movement in the 1960s and Asian immigration into the United States—and also illuminates how specific adjectives and occupations became more closely associated with certain populations over time. Our framework for temporal analysis of word embedding opens up a fruitful intersection between machine learning and quantitative social science.

in the large corpora of training texts (20–23). For a vector for the adjective honorable would be close to the man, whereas the vector for submissive would be close to woman. These stereotypes are automatically learned by the algorithm and could be problematic if the embedding is used for sensitive applications such as search rankings, recommendations, or translations. An important direction is to develop algorithms to debias the word embedding.

In this paper, we take another approach. We use word embeddings as a quantitative lens through which to study social trends—specifically trends in the gender and ethnic stereotypes in the 20th and 21st centuries in the United States. We develop a systematic framework and metrics to analyze word embeddings trained over 100 y of text corpora. We study the temporal dynamics of the word embedding capturing changes in gender and ethnic stereotypes over time. In particular, we quantify how specific biases decrease over time while others increase. Moreover, dynamics of the embedding

Language Modeling

Language Modeling

Q. What does it mean to "model something"?

- To mimic something
- To create a smaller/less complex version of something and study it
- To predict the future behavior of that something

"A good model would simulate the behavior of the real world: it would "understand" which events are in better agreement with the world, i.e., which of them are more likely."

- Lena Voita, NLP Course for you

Language Models

“ ... assign probabilities to sequences of words are called language models or LMs ”

- Jurafsky and Martin, Speech and Language Processing, Third Edition

Language Models

“ ... assign probabilities to sequences of words are called language models or LMs ”

- Jurafsky and Martin, Speech and Language Processing, Third Edition

city

in

place

water

example

“Mannheim is a beautiful ...”

Language Models: Surprisingly versatile

- Spelling and grammar correction
- Speech recognition
- Autocomplete and search
- Large Language Models
 - LLMs are trained to predict words
 - Left-to-right (**autoregressive**) LMs learn to predict next word

Language Modeling: Probability + Language

$P(\text{I saw a cat on } \dots) =$

$P(\text{I}) \cdot P(\text{saw}|\text{I}) \cdot P(\text{a}|\text{I saw}) \cdot P(\text{cat}|\text{I saw a}) \cdot P(\text{on}|\text{I saw a cat}) \cdot \dots$

Probability of **I saw a cat on**



Language Modeling: Probability + Language

$P(\text{I saw a cat on } \dots) =$

$$P(\text{I}) \cdot P(\text{saw}|\text{I}) \cdot P(\text{a}|\text{I saw}) \cdot P(\text{cat}|\text{I saw a}) \cdot P(\text{on}|\text{I saw a cat}) \cdot \dots$$

Probability of **I saw a cat on**

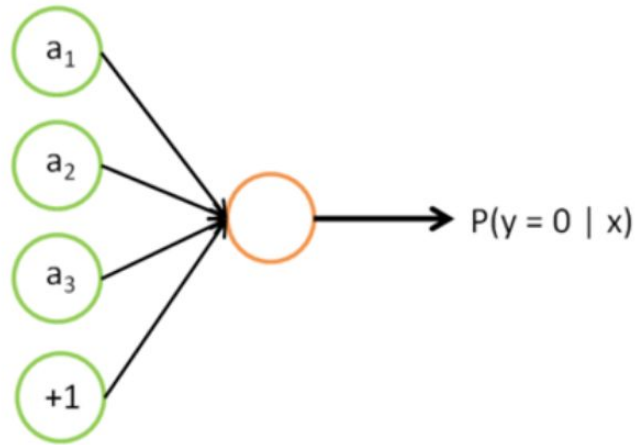


Formally, let $y_1, y_2, \dots, y_n$ be tokens in a sentence, and $P(y_1, y_2, \dots, y_n)$ the probability to see all these tokens (in this order). Using the product rule of probability (aka the chain rule), we get

$$P(y_1, y_2, \dots, y_n) = P(y_1) \cdot P(y_2|y_1) \cdot P(y_3|y_1, y_2) \cdot \dots \cdot P(y_n|y_1, \dots, y_{n-1}) = \prod_{t=1}^n P(y_t|y_{<t}).$$

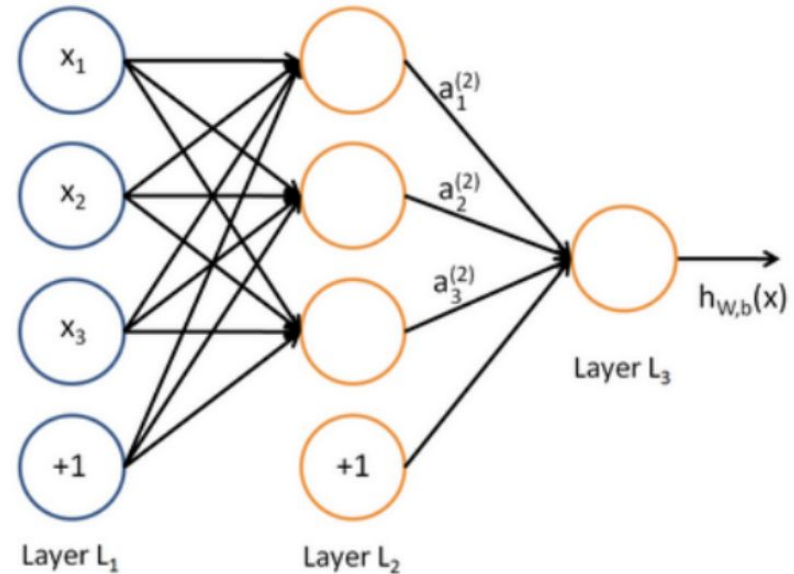
Quick detour: deep learning

Detour: Neural Networks and Deep Learning



Input
(features) Logistic
classifier

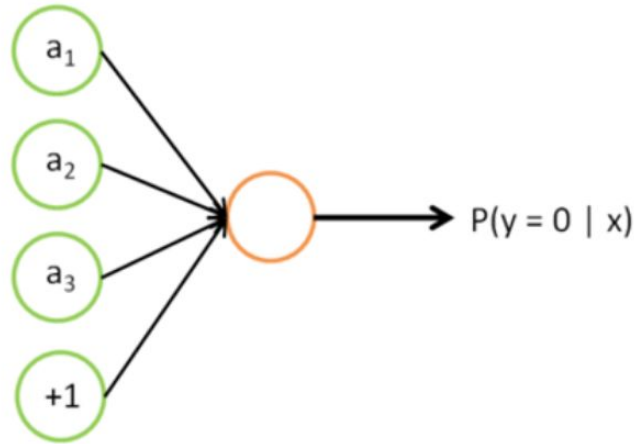
Logistic Regression



Neural Network

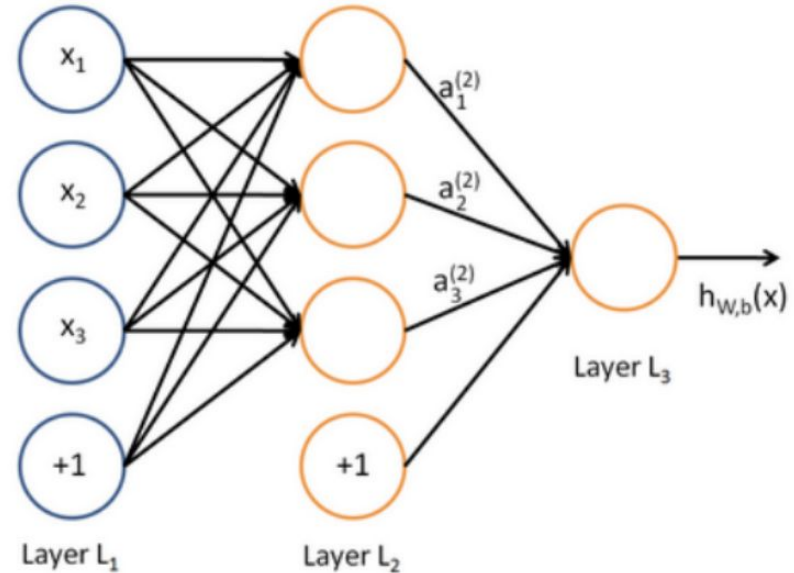
Detour: Neural Networks and Deep Learning

No need for feature engineering, the network learns higher or lower dimensions on itself



Input
(features) Logistic
classifier

Logistic Regression



Neural Network

Detour: Neural Networks and Deep Learning

What do we mean by feature engineering?

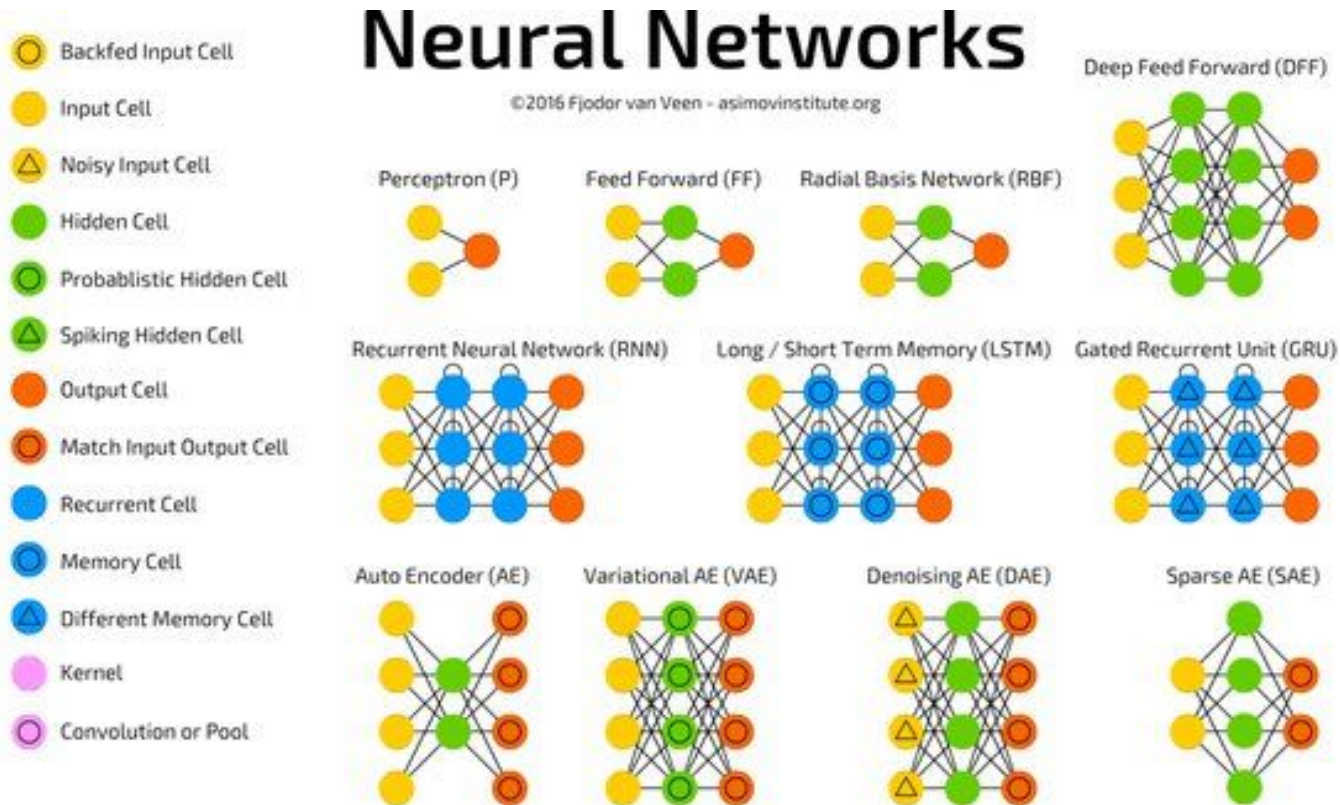
Say for sentiment analysis,

Table II. Summary of the Articles Employed as Supervised Method to Address TSA

Study	Task	Algorithms	Features	Dataset
Go et al. [2009]	TSA	NB, MaxEnt, SVM	unigrams, bigrams, POS	STS
Pak and Paroubek [2010]	TSA	MNB, SVM, CRF	unigrams, bigrams, trigrams, POS	own
Barbosa and Feng [2010]	TSA	SVM	meta-features (POS, polarity-MPQA), tweet syntax (i.e., retweet, hashtags, emoticons, links etc.)	own
Davidov et al. [2010]	TSA	kNN	word and n-gram based, punctuation-based, pattern-based	OC
Bakliwal et al. [2012]	TSA	SVM, NB	words' polarity, unigrams, bigrams, emoticons, hashtags, URLs, targets etc.	STS, Mejaj [Bora 2012]
Mohammad et al. [2013]	TSA	SVM	word/character n-grams, POS, caps, lexicons, punctuation, negation, tweet-based	SemEval-2013
Kiritchenko et al. [2014]	TSA	linear kernel SVM, MaxEnt	word/character n-grams, POS, caps, punctuation, emoticons, automatic sentiment lexicons, polarity, emphatic lengthening	SemEval-2013
Asiaee et al. [2012]	TSA	dictionary learning, WSVM, NB, kNN, SVM		DETC
Agarwal et al. [2011]	TSA	SVM	POS, unigrams, DAL lexicon, caps, exclamation etc.	own
Aisopos et al. [2011]	TSA	MNB, C4.5 tree	n-grams	own
Kouloumpis et al. [2011]	TSA	AdaBoost	N-gram with lexicon features, twitter-based, POS	STS, ETC
Saif et al. [2012b]	TSA	NB	unigrams, POS, sentiment-topic, semantic features	STS, HCR, OMD
Hamdan et al. [2013]	TSA	SVM, NB	unigrams, concepts (DBPedia), verb groups/adjectives (WordNet) and senti-features (SentiWordNet)	SemEval-2013
Jiang et al. [2011]	entity-TSA	SVM	unigrams, emoticons, hashtags, punctuation the General Inquirer lexicon	own
Aston et al. [2014]	TSA	Perceptron with Best Learning	character n-grams	Sanders

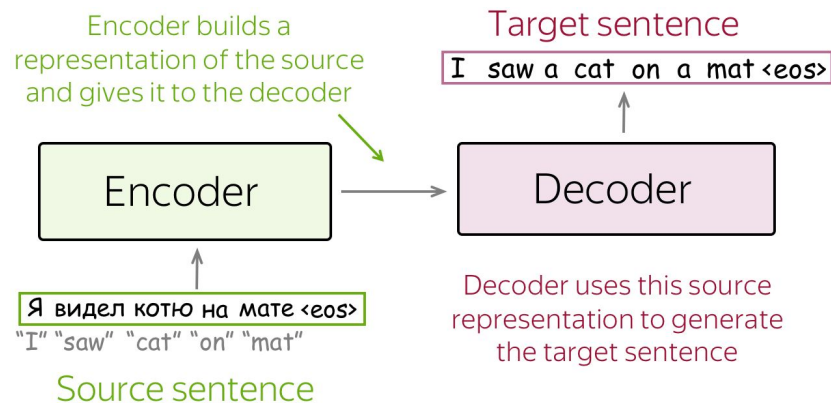
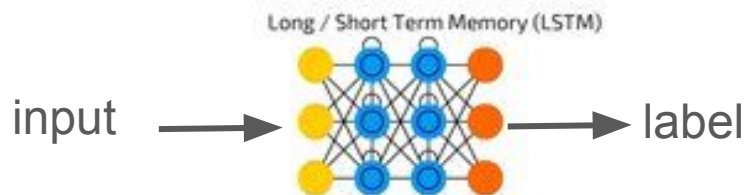
[Like It or Not: A Survey of Twitter Sentiment Analysis Methods](#), Anastasia Giachanou And Fabio Crestani, 2016

Detour: Neural Networks and Deep Learning



Deep Learning in NLP (till mid 2010's)

- Precludes need for explicit feature engineering
- Automatically creates complex representations
- Popular Classifiers are RNNs (GRUs and LSTMs)
- Classification is one strand of problems
- Another: sequence-to-sequence output, e.g., Machine Translation



Attention

Encoder builds a
representation of the source
and gives it to the decoder

Encoder

Я видел котю на мате <eos>

"I" "saw" "cat" "on" "mat"

Source sentence

Target sentence

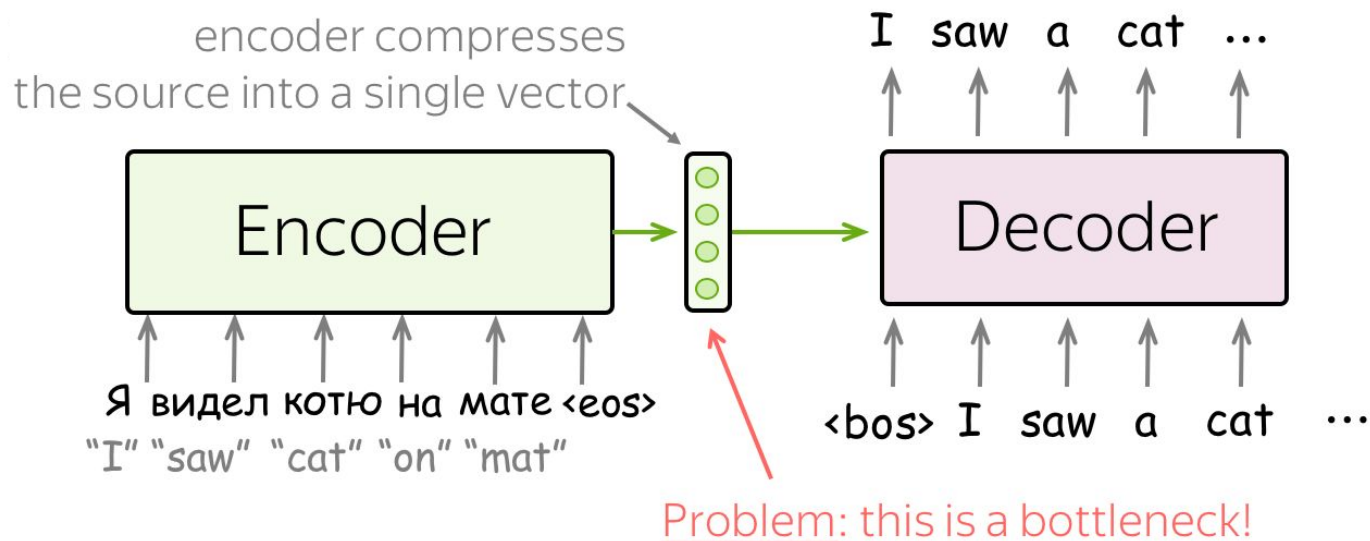
I saw a cat on a mat <eos>

Decoder

Decoder uses this source
representation to generate
the target sentence

Attention

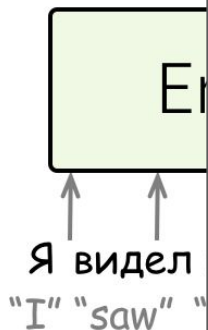
The fixed encoding issue



Attention

The fixed encoding

enco
the source int



Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "[Neural machine translation by jointly learning to align and translate.](#)" *arXiv preprint arXiv:1409.0473* (2014).

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

Jacobs University Bremen, Germany

Kyunghyun Cho **Yoshua Bengio***

Université de Montréal

ABSTRACT

Neural machine translation is a recently proposed approach to machine translation. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search

Attention

The fixed encoding

enco

Published as a conference paper at ICLR 2015

NEURAL MACHINE TRANSLATION BY JOINTLY LEARNING TO ALIGN AND TRANSLATE

Dzmitry Bahdanau

The idea behind 'Attention': At different steps, let a model "focus" on different parts of the input.

Я видел
"I" "saw"

tion. Unlike the traditional statistical machine translation, the neural machine translation aims at building a single neural network that can be jointly tuned to maximize the translation performance. The models proposed recently for neural machine translation often belong to a family of encoder-decoders and encode source sentence into a fixed-length vector from which a decoder generates a translation. In this paper, we conjecture that the use of a fixed-length vector is a bottleneck in improving the performance of this basic encoder-decoder architecture, and propose to extend this by allowing a model to automatically (soft-)search

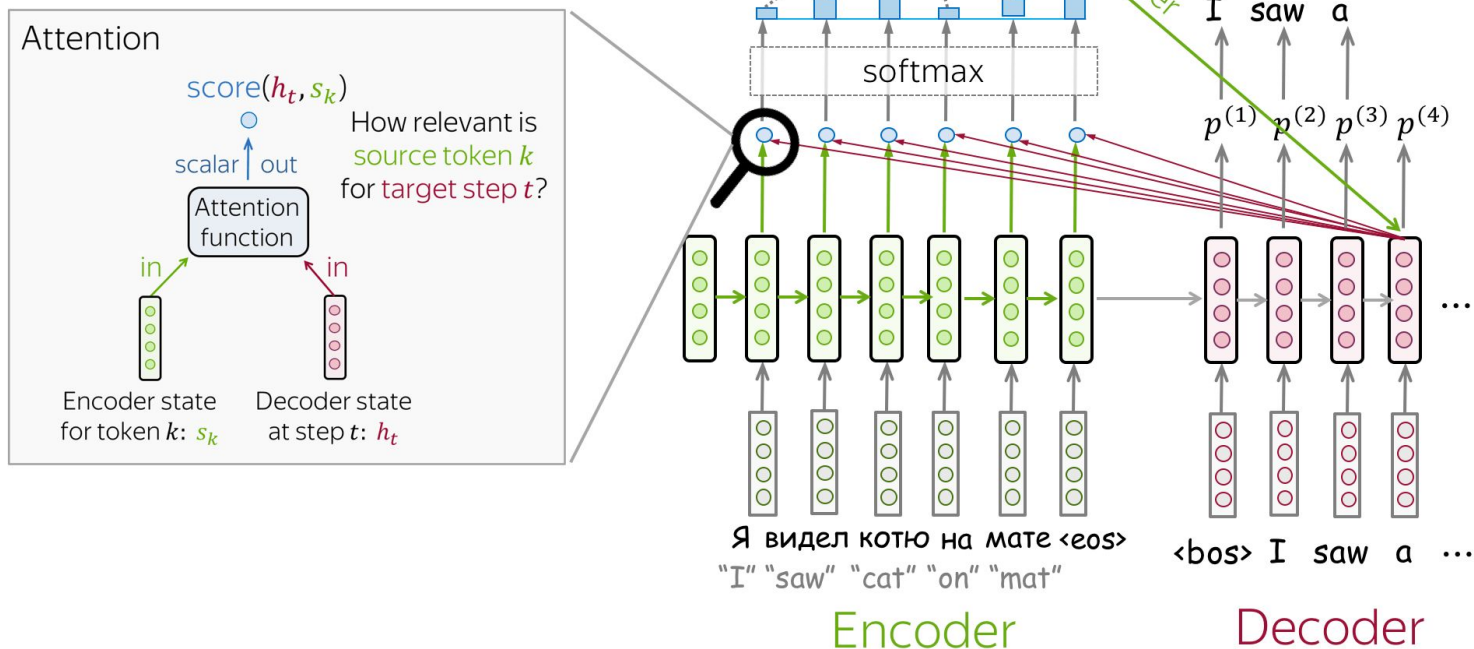
Bahdanau, Dzmitry, Kyunghyun Cho, and Yoshua Bengio. "[Neural machine translation by jointly learning to align and translate.](#)" *arXiv preprint arXiv:1409.0473* (2014).

Attention

Attention output: weighted sum of encoder states with attention weights

Attention weights: distribution over source tokens

A model can learn to “pay attention” to the most relevant source tokens for each step



Attention

	Seq2seq without attention	Seq2seq with attention
processing within encoder	RNN/CNN	RNN/CNN
processing within decoder	RNN/CNN	RNN/CNN
decoder-encoder interaction	static fixed- sized vector	attention

Contextual Embeddings

Contextualized Embeddings (ELMo, CoVe)

Deep contextualized word representations

Matthew E. Peters[†], Mark Neumann[†], Mohit Iyyer[†], Matt Gardner[†],
{matthewp, markn, mohiti, mattg}@allenai.org

Christopher Clark*, Kenton Lee*, Luke Zettlemoyer^{†*}
{csquared, kentonl, lsz}@cs.washington.edu

[†]Allen Institute for Artificial Intelligence

*Paul G. Allen School of Computer Science & Engineering, University of Washington

Abstract

Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, and Luke Zettlemoyer. 2018. [Deep Contextualized Word Representations](#). In *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*

deep contextual-
nat models both (1)
complex characteristics of word use (e.g., syn-
how these uses
contexts (i.e., to model
ers are learned func-
tions of the internal states of a deep bidirec-
tional language model (biLM), which is pre-
trained on a large text corpus. We show that
these representations can be easily added to
existing models and significantly improve the
state of the art across six challenging NLP

guage model (LM) objective on a large text cor-
pus. For this reason, we call them ELMo (Em-
beddings from Language Models) representations.
Unlike previous approaches for learning contextu-
alized word vectors (Peters et al., 2017; McCann
et al., 2017), ELMo representations are deep, in
the sense that they are a function of all of the in-
ternal layers of the biLM. More specifically, we
learn a linear combination of the vectors stacked
above each input word for each end task, which
markedly improves performance over just using
the top LSTM layer.

Contextualized Embeddings (ELMo, CoVe)

Deep contextualized word representations

Matthew E. Peters[†], Mark Neumann[†], Mohit Iyyer[†], Matt Gardner[†],
{matthewp, markn, mohiti, mattg}@allenai.org

Christopher Clark*, Kenton Lee*, Luke Zettlemoyer^{†*}

The idea behind Contextualized
Embeddings: Word *token* vectors instead of
word *type*, with neural networks modeling
left and right contexts

the 2018 Conference of the North American Chapter of the
Association for Computational Linguistics: *Human Language*
Technologies

contexts (i.e., to model
relationships between
tokens of the internal states of a deep bidirectional language model (biLM), which is pre-trained on a large text corpus. We show that these representations can be easily added to existing models and significantly improve the state of the art across six challenging NLP

et al., 2017). ELMo representations are deep, in the sense that they are a function of all of the internal layers of the biLM. More specifically, we learn a linear combination of the vectors stacked above each input word for each end task, which markedly improves performance over just using the top LSTM layer.

Contextualized Embeddings (ELMo, CoVe)



Deep contextualized word representations

Matthew E. Peters[†], Mark Neumann[†], Mohit Iyyer[†], Matt Gardner[†],
{matthewp, markn, mohiti, mattg}@allenai.org

Christopher Clark^{*}, Kenton Lee^{*}, Luke Zettlemoyer^{†*}

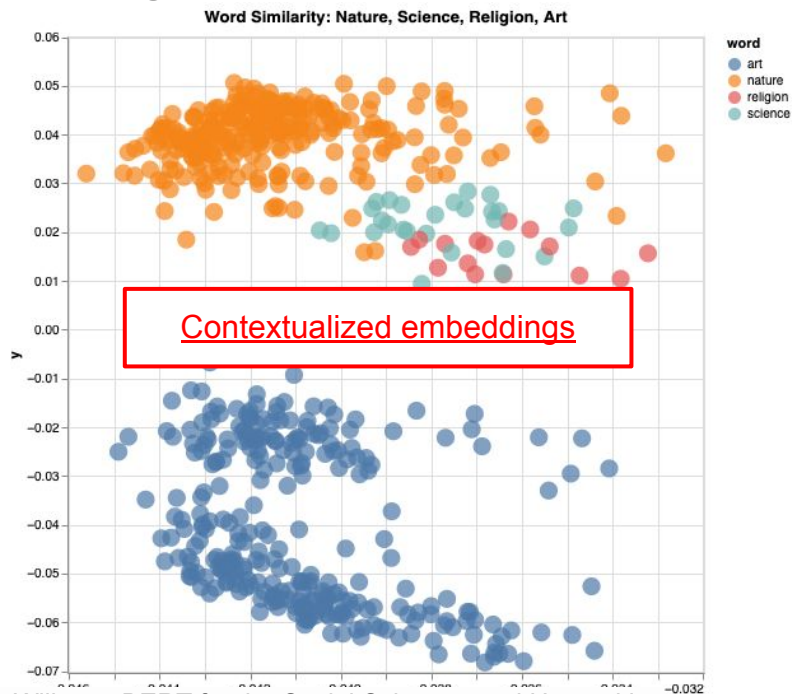
Behind Contextualized
Embeddings. word *token* vectors instead of
word *type*, with neural networks modeling
left and right contexts

vary across linguistic contexts (i.e., to model polysemy). Our word vectors are learned functions of the internal states of a deep bidirectional language model (biLM), which is pre-trained on a large text corpus. We show that these representations can be easily added to existing models and significantly improve the state of the art across six challenging NLP

et al., 2017). ELMo representations are deep, in the sense that they are a function of all of the internal layers of the biLM. More specifically, we learn a linear combination of the vectors stacked above each input word for each end task, which markedly improves performance over just using the top LSTM layer.

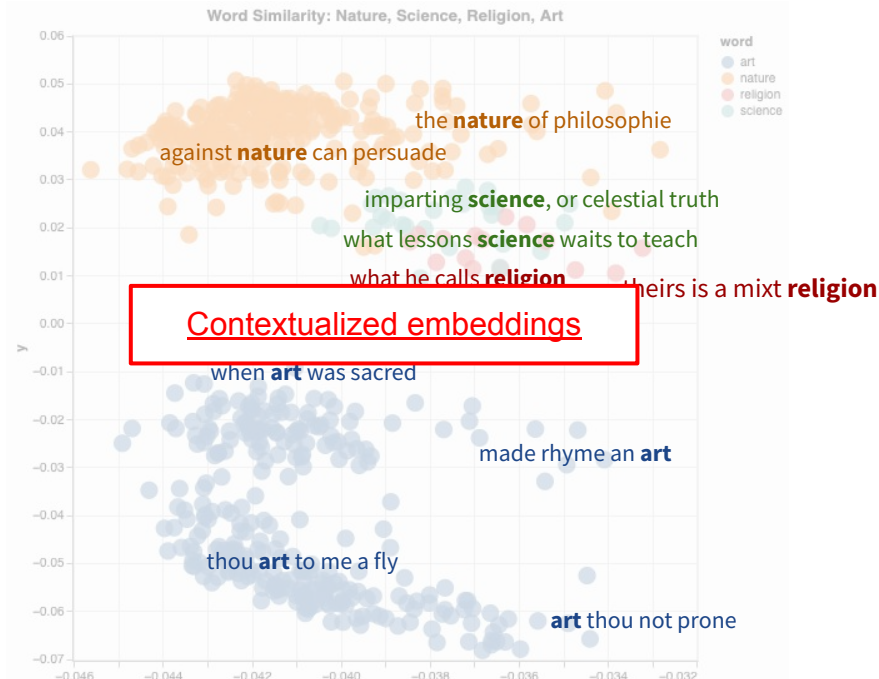
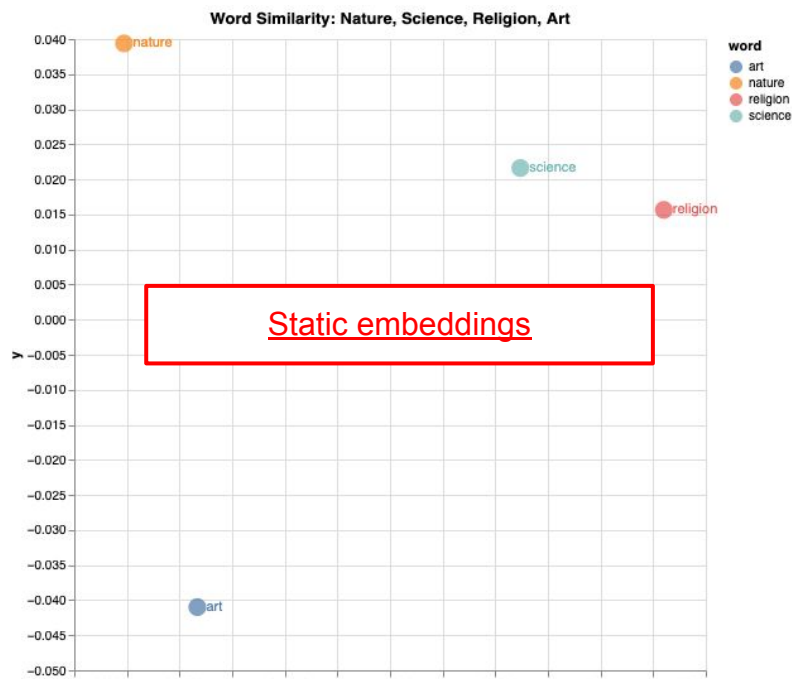
Contextualized Embeddings (ELMo, CoVe)

Result: multiple vectors of a word depending on its context



Contextualized Embeddings (ELMo, CoVe)

Result: multiple vectors of a word depending on its context



Transformers

Transformers

- RNNs in architectures have limited context
- Fix: replace all components with attention

	Seq2seq without attention	Seq2seq with attention	Transformer
processing within encoder	RNN/CNN	RNN/CNN	attention
processing within decoder	RNN/CNN	RNN/CNN	attention
decoder-encoder interaction	static fixed- sized vector	attention	attention

Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

Llion Jones*
Google Research
llion@google.com

Aidan N. Gomez* †
University of Toronto
aidan@cs.toronto.edu

Lukasz Kaiser*
Google Brain
lukaszkaizer@google.com

Illia Polosukhin* ‡
illia.polosukhin@gmail.com

Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including

Vaswani, Ashish, et al. "[Attention is all you need.](#)" *Advances in neural information processing systems* 30 (2017).

Transformers

Attention Is All You Need

Ashish Vaswani*
Google Brain
avaswani@google.com

Noam Shazeer*
Google Brain
noam@google.com

Niki Parmar*
Google Research
nikip@google.com

Jakob Uszkoreit*
Google Research
usz@google.com

The idea behind ‘Transformers’: Replace all components with Attention to model context better

.com

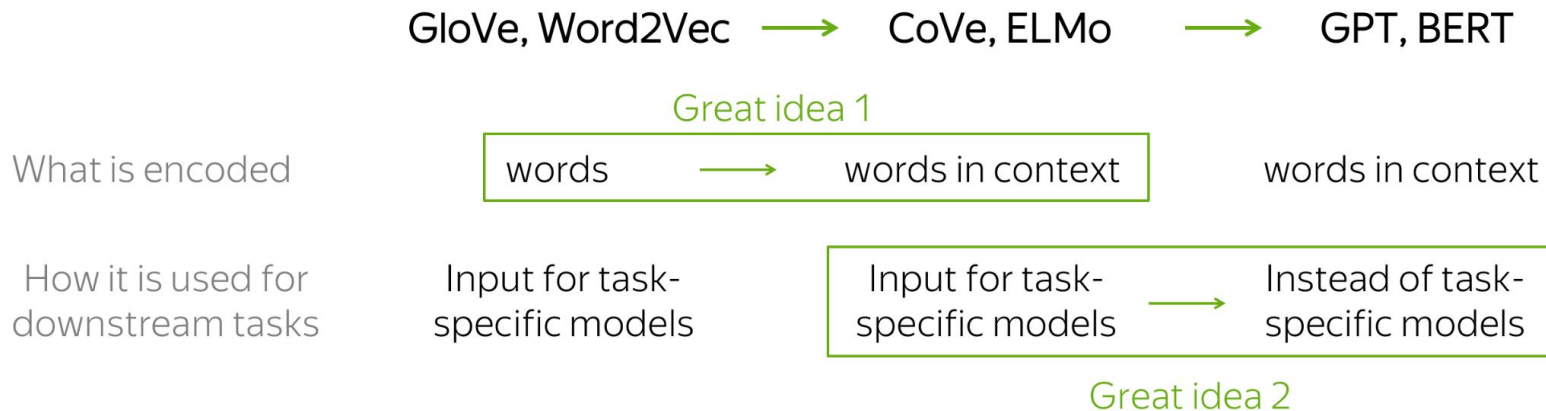
Abstract

The dominant sequence transduction models are based on complex recurrent or convolutional neural networks that include an encoder and a decoder. The best performing models also connect the encoder and decoder through an attention mechanism. We propose a new simple network architecture, the Transformer, based solely on attention mechanisms, dispensing with recurrence and convolutions entirely. Experiments on two machine translation tasks show these models to be superior in quality while being more parallelizable and requiring significantly less time to train. Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task, improving over the existing best results, including

Vaswani, Ashish, et al. "[Attention is all you need.](#)" *Advances in neural information processing systems* 30 (2017).

One model to rule them all: Pretraining and Fine-tuning

BERT, GPT (the first one),



BERT, GPT (the first one),

G

What is encoded

How it is used for
downstream tasks

Devlin, Jacob, et al. "[Bert: Pre-training of deep bidirectional transformers for language understanding.](#)" *Proceedings of the 2019 conference of the North American chapter of the association for computational linguistics: human language technologies, volume 1 (long and short papers)*. 2019.

BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding

Jacob Devlin Ming-Wei Chang Kenton Lee Kristina Toutanova

Google AI Language

{jacobdevlin, mingweichang, kentonl, kristout}@google.com

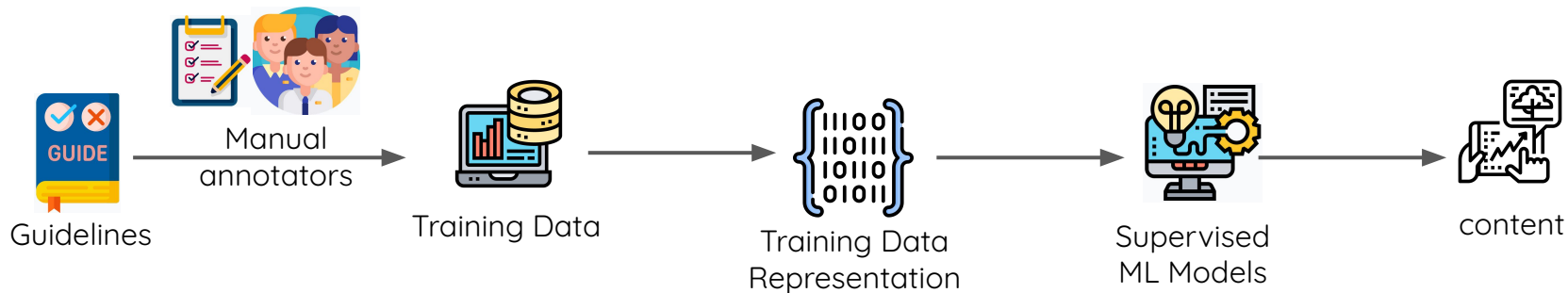
Abstract

We introduce a new language representation model called **BERT**, which stands for **B**idirectional **E**ncoder **R**epresentations from **T**ransformers. Unlike recent language representation models (Peters et al., 2018a; Radford et al., 2018), BERT is designed to pre-train deep bidirectional representations from unlabeled text by jointly conditioning on both left and right context in all layers. As a result, the pre-trained BERT model can be fine-tuned with just one additional output layer to create state-of-the-art models for a wide

There are two existing strategies for applying pre-trained language representations to downstream tasks: *feature-based* and *fine-tuning*. The feature-based approach, such as ELMo (Peters et al., 2018a), uses task-specific architectures that include the pre-trained representations as additional features. The fine-tuning approach, such as the Generative Pre-trained Transformer (OpenAI GPT) (Radford et al., 2018), introduces minimal task-specific parameters, and is trained on the downstream tasks by simply fine-tuning *all* pre-trained parameters. The two approaches share the

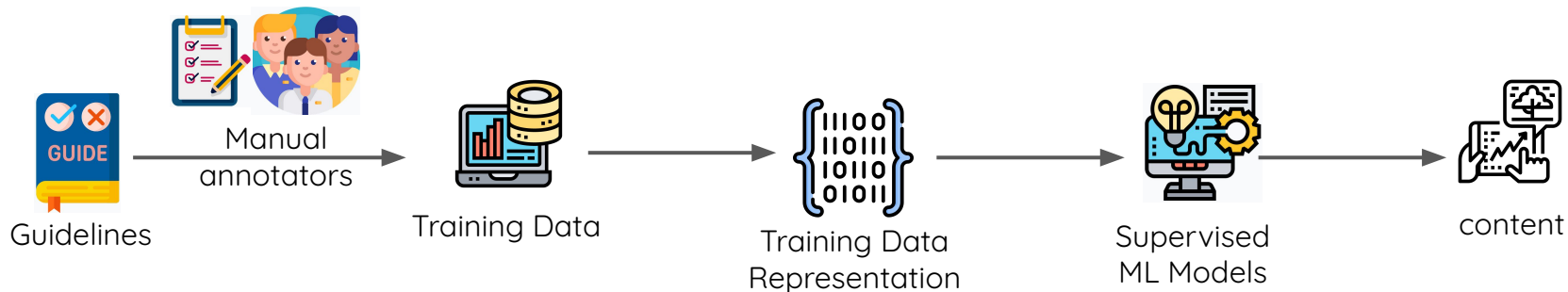
Revisiting Machine Learning

- Usual workflow, train model on your dataset for a certain task.
- The dataset has to represent exactly what task you want to do

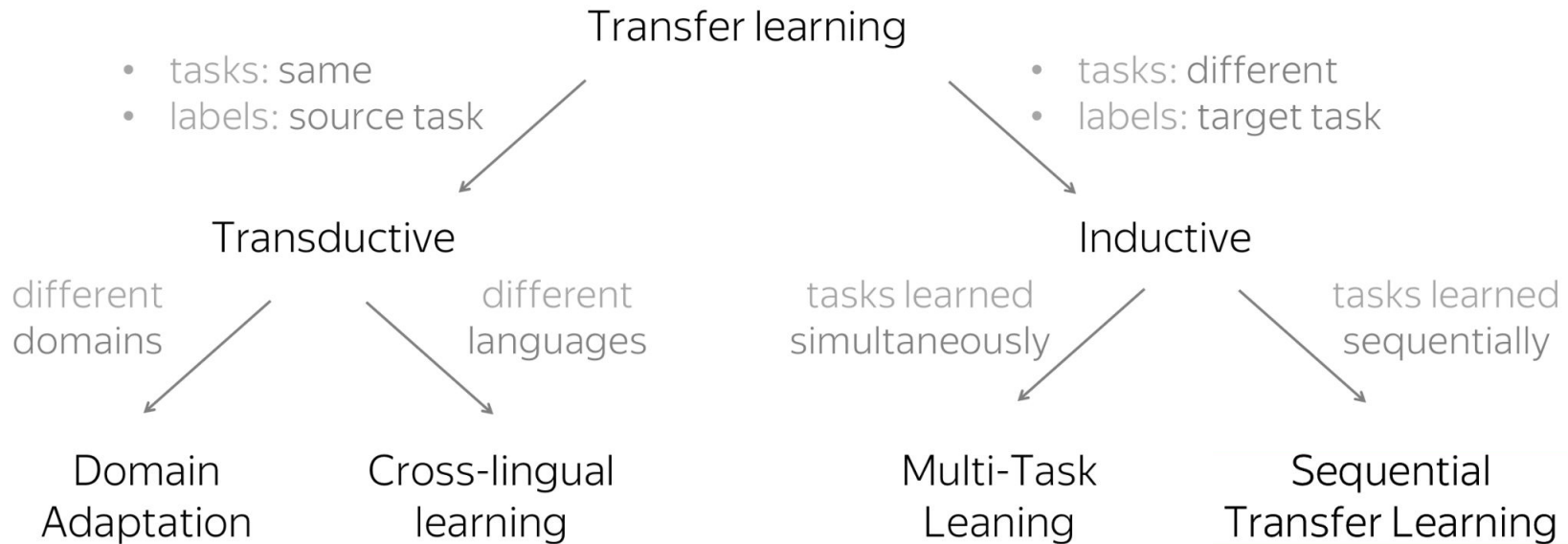


Revisiting Machine Learning

- Usual workflow, train model on your dataset for a certain task.
- The dataset has to represent exactly what task you want to do
- However,
 - Sometimes, you do not have enough labeled data for a very similar task
- Transfer Learning!



Revisiting Machine Learning: Transfer Learning



This taxonomy is from [Sebastian Ruder's blog post](#).

The old vs. 'new' paradigm (maybe revisited due to LLMs)

OLD: Train a model on a task where we have labeled data

NEW:

1. **Pre-train** a model on a task where we have lots of data
 - *e.g., we have lots and lots of internet data*
2. **Fine-tune** the model on your downstream task
 - *e.g., a specific, curated dataset that you're studying*

 **Transfer learning!**

Transfer Learning via Pretraining

- Train from scratch

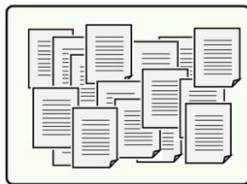
What they will know:



May be not enough to
learn relationships
between words

- Take pretrained
(Word2Vec, GloVe)

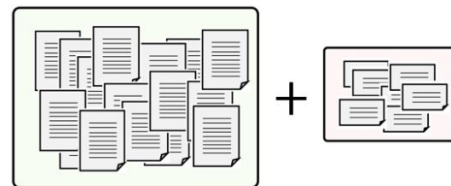
What they will know:



Know relationships between words,
but are **not** specific to the task

- Initialize with pretrained,
then fine-tune

What they will know:



Know relationships between
words and adapted for the task

“**Transfer**” knowledge from a huge unlabeled corpus to your task-specific model

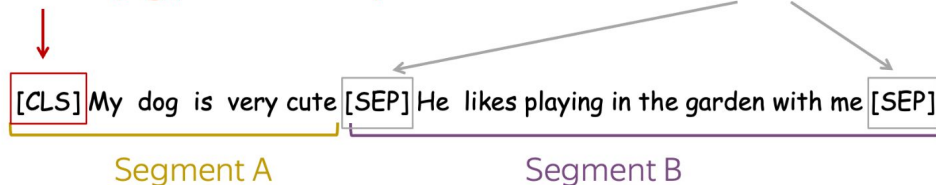
How is BERT Pre-trained?

- Pre-training Input: Pairs of Sentences with Special Tokens separating them

[CLS]: Special token

- Training time: predict if sentences are consecutive or not (Next Sentence Prediction /NSP objective)
- Test time: downstream tasks (e.g., classification)

[SEP]: Special token-separator



Training on pairs of sentences: either consecutive or random (50%/50%)

- Pre-training Objective:

- Next Sentence Prediction (NSP)
- Masked Language Modeling (MLM)

- Lena Voita, NLP Course for you

Massive and Undocumented Pretraining Sets

What kind of datasets are used to pre-train models like BERT?

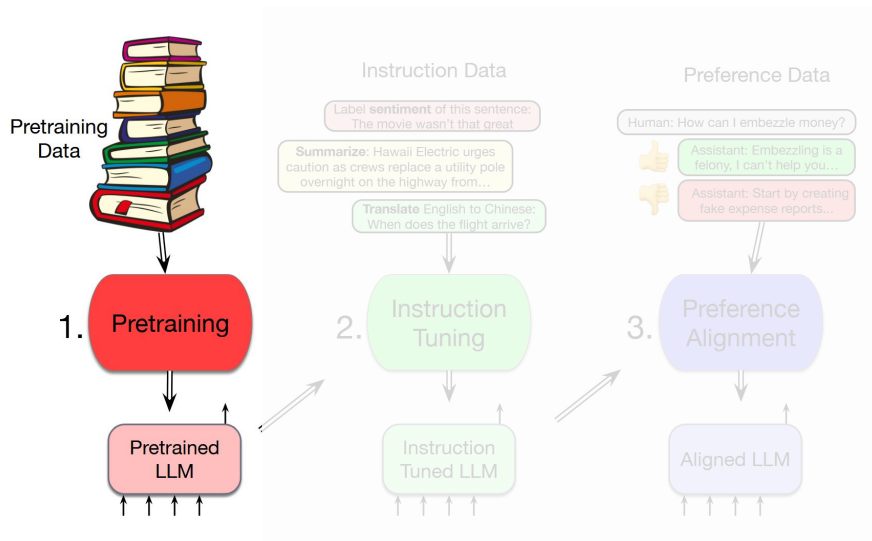
- “BooksCorpus”
- English Wikipedia
- “The Pile”
- “Colossal Clean Crawled Corpus”

What is fine-tuning?

- Inputs:
 - pretrained model (usually a transformer, BERT and variants are still very popular)
 - Labeled dataset (rule-of-thumb, at least 1K examples)
- Adds a new layer containing classification parameters.
- All model parameters are updated to maximize the log probability of the labels.
- Output: classifier for your specific task

We now have all the ingredients for step 1

- Efficient representations: contextualized embeddings
- Neural language model
- Pretrained in an unsupervised way on large amounts of data
- BERT only has encoders
- Models that generate text also include a “decoder” block, e.g., GPT-2, GPT-3 (OpenAI)



Jurafsky and Martin, 2025

Logistics: More info on the projects

Group Project

- **Equivalent to a scientific short paper: 4-5 pages, double column**
- You have to work in teams of 2-5; team size suggestion will be finalized based on how many people register for the course
- If you cannot find project partners: use the forum on ILIAS
 - Worst case, you'll be merged with another group
- We have some suggestions for projects ideas, but you are encouraged to come up with your own and be creative!

More about the Projects: Timeline and Dates

- Start forming groups: week 2
- Project pitches: 6.10.2025 [5%]
- Midway Presentation: 27.10.2025 (during the lecture) [10%]
- Final presentation; 30.11.2025 (during one or both exercise sessions) [15%]
- Final reports due: 15.01.2026 [50%]

More about the Projects: Timeline and Dates

- Start forming groups: week 2

- Project pitches: 6.10.2025 [5%]

You cannot drop the course after the project pitches, so be sure to deregister earlier if you don't want to participate in the course

- Midway Presentation: 27.10.2025 (during the lecture) [10%]
- Final presentation; 30.11.2025 (during one or both exercise sessions) [15%]
- Final reports due: 15.01.2026 [50%]

More about the Projects: Timeline and Dates

- Start forming groups: week 2

- Project pitches: 6.10.2025 [5%]

You cannot drop the course after the project pitches, so be sure to deregister earlier if you don't want to participate in the course

- Midway Presentation: 27.10.2025 (during the lecture) [10%]

You also cannot change groups after the project pitches

- Final presentation; 30.11.2025 (during one or both exercise sessions) [15%]

- Final reports due: 15.01.2026 [50%]

More about the Projects: Presentations and Report

- Project pitches: 6.10.2025 [5%] → 5 mins per teams
- Midway Presentation: 27.10.2025 (during the lecture) [10%] → 10 mins
- Final presentation; 30.11.2025 (during one or both exercise sessions) [15%] → 20 mins
- Final reports due: 15.01.2026 [50%]
 - All reports should include: introduction, related work, experiments, results, conclusion
 - Team member contributions: all members of a group need not get the same grades
 - Ideally: also include your code

Project Grading Rubric: Report

Each report (50%) should consist of:

- **Abstract:** 250-word summary of your project including a formulation of its research questions and an overview of methods and results [5]
- **Motivation:** What question(s) do you seek to answer and why? Elaborate on how this research matters in terms of science, technology and/or society [5]
- **Research background:** What previous work is related to this project and what can we learn from it? Do not just cite papers but anything relevant and reflect about the cited contributions on your text [5]
- **Data:** Describe your data sources including descriptive statistics and plots. Carefully document all references to data sources. [10]

Project Grading Rubric: Report

- **Methods:** Explain how you filtered data, normalized values, computed additional variables, etc. Detail the statistical analyses and other methods you will apply to assess your research question(s). Carefully document all references to methods and packages used. [10]
- **Results:** Expose the results of your analysis including tables and figures that communicate and illustrate those results [10]
- **Conclusion and Discussion:** Evaluate answers to the question and their reliability. Identify limitations and alternative explanations for your results. [5]
- **Author contributions:** what each team member worked on [remember, all members need not get the same grade]

Project Grading Rubric

- **Methods:** Explain how you filtered data, normalized values, computed additional variables, etc. Detail the statistical analyses and other methods you will apply to assess your research question(s). Carefully document all references to methods and packages used. [10]
- **Results:** Expose the results of your analysis including tables and figures that communicate and illustrate those results [10]

Good results != positive findings, but rather
well-interpreted and transparent results.
- **Conclusion and Discussion:** ty. Identify limitations and alternative explanations for your results. [5]
- **Author contributions:** what each team member worked on [remember, all members need not get the same grade]

AI Usage Policy

- You have to report when and where you used Large Language Models for your project (for both the report and presentations), but only report them in the final report
- We do not prohibit the usage of LLMs, but you are responsible for all the contents in your report and project
 - If there are errors e.g., wrong statistics or incorrect references, that will count against you
- Use LLMs responsibly
 - As we will see in this course, they are very good at certain parts of the scientific processing (grammar-check, etc) but not so good at others (critical analysis, full project ideation, etc...)
- Do **not** offload your thinking to LLMs

Project Grading Rubric: Presentations [5 + 10 + 15]

- Students get questions from the audience (you, the instructors, and maybe some other researchers) regarding the topic and feedback on how to improve the project for the final report.
- Each presentation has to contain four parts:
 - a. Research question and motivation [5]
 - b. Data and methods [5]
 - c. Results [10] [only for the midterm and final presentation]
 - d. Conclusion and critique [5] [only for the final presentation]
- All members of the group have to participate in their presentation and answer questions.

Summary and Next Week

- This session we learned:
 - About the computational turn in the Social Sciences
 - About LLMs and how they contribute to the Social Sciences
 - The first building blocks of LLMs
 - Words → Tokens
 - Text Representations
 - Using text representations in text classification
- Next time:
 - Getting LLMs to generate
 - Getting LLMs to follow instructions
 - Getting LLMs to ‘behave’

Questions?

Additional Resources

- The Text Analytics course (IS 661)
- Daniel Jurafsky and James H. Martin. 2025. Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models, 3rd edition. Online manuscript released August 24, 2025.
<https://web.stanford.edu/~jurafsky/slp3>
 - https://web.stanford.edu/~jurafsky/slp3/slides/tokens_aug25.pdf
 - <https://web.stanford.edu/~jurafsky/slp3/slides/logreg25aug.pdf>
- Elena Voita's NLP course for you:
https://lena-voita.github.io/nlp_course.html
 - https://lena-voita.github.io/nlp_course/text_classification.html